GENESIS 2.5: opt-in OpenCL and CUDA acceleration for the GENESIS/PGENESIS compartmental neural simulator

Karol Chlasta[a,*], Grzegorz Marcin Wójcik[b]

[a] *Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al. Jana Pawła II 43A/37B, 01-001 Warsaw, Poland*
[b] *Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator; PGENESIS extends it to MPI. We release GENESIS 2.5, adding opt-in OpenCL and CUDA backends for the Hodgkin–Huxley channel-update path and `hines_tree_eliminate`, a GPU Hines tree-elimination kernel for axially-coupled dendritic trees, leaving every existing CPU, MPI, model and SLI-script workflow unchanged. On NVIDIA A40/A100 hardware it reaches 21× end-to-end speedup for single-compartment networks and 57× for 160 000-compartment populations at realistic run lengths, matching the fp64 CPU to $\sim 10^{-7}$ V. On the Vogels–Abbott COBAHH benchmark GENESIS 2.5 runs $1.63 \pm 0.07\times$ faster than CoreNEURON. The release also fixes seven latent defects, including silently wrong accelerator results, and reduces $O(n^2)$ model construction to linear.

Keywords: GENESIS, PGENESIS, OpenCL, CUDA, GPU acceleration, compartmental neural simulation, Hines solver

Required Metadata

Current code version

Main text

1. Motivation and significance

*Corresponding author.
Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl` (Grzegorz Marcin Wójcik)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.5 (2026-07-25; OpenCL + CUDA-validated, multi-compartment GPU Hines solve)
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/WarsawIQ/genesis-2.5` (archived release: `https://doi.org/10.5281/zenodo.21658389`)
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions) – see `LICENSE`
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, OpenCL C, CUDA C++ (accelerator backends), MPI (PGENESIS), Python (benchmark analysis/plotting)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64, GCC, GNU make, an OpenCL 1.2+ runtime (tested: ROCm 6.3.1 and Mesa rusticl) or a CUDA 12.x toolkit (tested: CUDA 12.8, `sm_80` NVIDIA A100 and `sm_86` NVIDIA A40 on the UMCS cluster) for the GPU backends, an MPI implementation for PGENESIS (MPICH/HYDRA or Open MPI tested on the UMCS cluster)
C7	If available Link to developer documentation/manual	`README.md` and `paper/docs/REPLICATION.md` in the repository
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

Compartmental simulators remain essential where channel kinetics and dendritic geometry cannot be abstracted away. GENESIS [1] is among the longest-maintained simulators in this class, with PGENESIS ex-

tending it to MPI; both are still used in production workflows, scripted in SLI, independent of newer ecosystems.

Modern workstations ship capable integrated GPUs, but GENESIS has no accelerator backend: every channel-kinetics update runs on the CPU regardless of available hardware, though per-compartment gating integration is embarrassingly parallel. Existing GENESIS/PGENESIS users have no path to accelerator support without abandoning their models and SLI scripts.

Existing accelerated simulators, and what they ask of a user. Compartmental simulation on GPUs is not new, and the alternatives are mature. NEURON [11] is the most widely used simulator in this class; CoreNEURON [8] is its optimized compute engine, which strips the interpreter and runs the same models on GPUs through OpenACC, reaching large speedups on multi-rank and multi-GPU systems. Arbor [7] was written from scratch for many-core and GPU hardware, with a performance-portable back end and its own model description. For network models of point neurons rather than detailed morphologies, NEST [12] targets large-scale distributed simulation, and Brian 2 [13] generates code from equations, with GPU execution available through GeNN [14].

Each of these is a strong choice for a new model. None is a path for an existing GENESIS model. CoreNEURON accelerates NEURON's models, not GENESIS's; Arbor and Brian 2 require the model to be re-expressed in their own descriptions; and a GENESIS user's investment is typically in SLI scripts, `.p` cell files and channel prototypes accumulated over years. Porting is not a mechanical translation — as this work found, two implementations of the same published benchmark differ enough in channel kinetics and connectivity construction that establishing their equivalence is itself experimental work (Section 3). The gap GENESIS 2.5 addresses is therefore not "fastest simulator" but "acceleration without migration": the same models and scripts, unchanged, on the hardware the user already has.

Contributions. This work contributes:

1.1 Opt-in OpenCL and CUDA backends for the `hsolve` channel-update path, behind one entry point, leaving every existing CPU, MPI, model and SLI-script workflow unchanged.

1.2 `hines_tree_eliminate`, a GPU tridiagonal (Hines [10]) elimination kernel extending acceleration from isopotential cells to axially-coupled dendritic trees, running the identical elimination the CPU solver does.

1.3 Seven defect fixes: four in the accelerator, which produced silently wrong results, and three in the Hines solver, which suppressed `inject`-driven transients in any multi-neuron `hsolve`. The latter affect existing users regardless of GPU use.
1.4 A latent $O(n^2)$ model-construction bottleneck, unrelated to accelerator support, reduced to linear.
1.5 A head-to-head evaluation against NEURON and CoreNEURON on a published network model, in which we verified that the two implementations are equivalent, and timed both arms the same way.

GENESIS 2.5 closes this gap: GENESIS 2.4/PGENESIS 2.4 plus an OpenCL backend for the `hsolve` channel-update path, and a CUDA backend on the same interface (a faithful fp32 port, validated on UMCS cluster NVIDIA A100/A40 nodes; Section 3). No existing CPU, MPI, model, or SLI-script workflow needs to change. While validating this backend we additionally found and fixed three independent solver defects in `hines_solve.c`/`hines_init.c` affecting any multi-neuron `hsolve` configuration: current injection (`inject`) silently failed to produce a voltage response prior to the fix, which would have invalidated any dynamics-comparing benchmark and may have affected unrelated multi-neuron `hsolve` models elsewhere.

Why a GPU helps here at all is specific to the workload. In a Hodgkin–Huxley model the per-compartment gating-variable integration is the dominant cost, and it is embarrassingly parallel: each compartment's channel state advances from its own voltage and its own table lookups, with no coupling to any other compartment within the step. The state involved is small and the arithmetic intensity low, so the work maps onto thousands of lightweight threads without the memory pressure that limits many scientific kernels. That makes the channel-update path the natural first target, ahead of the tridiagonal solve.

We provide both OpenCL and CUDA rather than choosing one because they buy different things. OpenCL is vendor-neutral and runs on the integrated GPUs now shipped with ordinary workstations, including devices without double-precision support, which is why the kernel is written in fp32 throughout. CUDA targets the NVIDIA datacenter and desktop cards where most cluster time is available and where the toolchain and profiling tools are standard. Both sit behind one entry point, so a model and its SLI script are unchanged between them and the choice is a build flag.

We name this release "2.5" rather than "3.0" deliberately: "GENESIS 3" [2] produced independent successor lines (Neurospaces/Heccer [3],

MOOSE) evolving into the CBI federation [4, 5, 6], not a single artifact comparable to GENESIS 2.4. GENESIS 2.5 is not a re-architecture and targets existing GENESIS 2.4 deployments that should not need to migrate models or scripts to gain accelerator support.

2. Software description

   2.1 Software architecture

   GENESIS 2.5 leaves the existing architecture (SLI interpreter, `hsolve` solver, PGENESIS MPI partitioning) unmodified and adds one component: an OpenCL backend invoked from `hsolve` when an attached element uses `chanmode=4` (or 5) with real ion-channel state. Two dispatch modes are implemented:

   - *Per-step dispatch* (`ocl_chip_update`): one `clEnqueueNDRangeKernel` call per simulation step, with channel state uploaded and downloaded every step.
   - *Multiloop dispatch* (`chip_channel_multiloop`, via `GENESIS_OCL_MULTILOOP=<K>`): a single kernel dispatch runs all K steps in an inner GPU-side loop, amortizing the per-step PCIe transfer cost that otherwise dominates wall time (Section 3).

   Every accelerator run emits a non-empty profiling summary, which guards against two silent failures: an `hsolve` with no attached channels never reaching the kernel, and a kernel that fails to build falling back to CPU unnoticed. The kernel is fp32 throughout, so it runs on runtimes without double-precision support; the rest of `hsolve` stays `double`, converting only at the buffer boundary.

   Both dispatch modes update each compartment independently, exact only for single-compartment (isopotential) neurons: a real dendritic tree's compartments are coupled through axial resistance, solved on the CPU by `hsolve` as a compiled bytecode program performing a bottom-up/top-down Gaussian elimination over the tree's tridiagonal system once per step. We add a third kernel, `hines_tree_eliminate` (`ocl_channel.cl`; CUDA port in `cuda/cuda_channel.cuh`), executing this identical bytecode on the GPU, one thread per disconnected tree (neuron), operating on the same flat opcode/coefficient arrays the CPU solver already builds at `SETUP` time. Because independent trees never reference each other's rows (the combined program is block-diagonal), threads need no synchronization. The channel and elimination kernels dispatch back to back on the same queue/stream, so the K-step batch still uploads and downloads state once. `ocl_hsolve.c`/`cuda_hsolve.c` select this path whenever the `hsolve` contains

a multi-compartment tree, leaving the original kernel for single-compartment networks.

2.2 Software functionalities

- Unmodified serial (`genesis`), headless (`nxgenesis`), and MPI (PGENESIS) execution paths.
- OpenCL channel-update kernel for Hodgkin–Huxley-style `tabchannel` gating (Na X^3Y^1, K X^4), dispatched per-step or batched via multiloop.
- A CUDA backend at the same call site (`genesis/src/hines/cuda/`): a line-for-line fp32 port behind an identical entry point, so one model and harness drive either backend.
- Device-side dispatch confirmation distinguishing a genuine GPU run from CPU fallback.
- A driven-spiking benchmark (`hh_spiking_benchmark.g`) exercising the repaired `inject` path.
- Three corrected Hines-solver defects (forward-elimination fast-path guard, backward-elimination root handling, `SET_DIAG`/`SKIP_DIAG` row selection), independent of GPU support.
- `hines_tree_eliminate`, a GPU tridiagonal elimination kernel in both backends, extending acceleration to axially-coupled dendritic trees (Section 3).
- Five corrected $O(n^2)$ model-construction defects, restoring linear-time construction independent of GPU support (Section 3).

The tridiagonal (Hines) elimination kernel is what decides whether any of this reaches realistic morphologies. Accelerating channel updates alone leaves the axial coupling between compartments on the CPU, so a multi-compartment model gains little: the solve becomes the bottleneck as soon as the channel work is removed from it. `hines_tree_eliminate` moves that solve onto the device as well, running the identical elimination the CPU performs, one thread per neuron. The consequence is a change in kind rather than degree — acceleration stops being limited to isopotential point-neuron networks and applies to populations of real dendritic trees, which is the regime Section 3 measures.

3. Illustrative examples

Reproducing the CPU baseline. `nxgenesis` on the bundled `Cal8.g` and `Cal7difshell.g` models gives stable, low-variance timings (CV $\approx 1\%$, 30 replicates), confirming ordinary GENESIS workflows are unaffected by this release.

Reproducing the GPU multiloop result. `genesis/Scripts/benchmark/hh_spiking_benchmark.g` builds N driven, spiking Hodgkin–Huxley neurons under one `hsolve` (`chanmode=4`); `GENESIS_OCL_MULTILOOP=`K dispatches all K steps in one GPU call. Both arms (`nxgenesis_nocl` fp64 CPU; `nxgenesis` fp32 multiloop, AMD Radeon 890M) are timed identically at $K = 50\,000$ and $K = 0$, isolating the *simulation phase* from construction; 30+ reps, 95% CIs in `experiments/data/campaign_wallclock_summary.csv`.

Table 2 and Fig. 1 report the result after the scheduler and solver fixes. Step-phase speedup peaks near $N = 5000$ at ∼18× sustained and ∼13.5× cold-start (∼1.5×10^9 neuron-steps/s); the gap reflects the idle, downclocked GPU during construction. Beyond $N \approx 10\,000$ the integrated GPU no longer sustains its fast clock state and speedup falls toward ∼3×. End-to-end speedup peaks at ∼4× and approaches the step-phase figure as simulations lengthen.

N	**CPU step (s)**	**GPU step (s)**	**Step-phase sustained**	**Step-phase single-run**	**Throughput (M nsteps/s)**
500	0.302	0.112	2.7×	2.0×	223
1 000	0.587	0.112	5.2×	3.7×	446
2 000	1.166	0.117	10.0×	6.5×	855
5 000	2.965	0.164	**18.1**×	13.5×	**1524**
10 000	5.799	0.406	14.3×	6.8×	1232
20 000	11.918	3.283	3.6×	3.1×	305
50 000	31.179	9.880	3.2×	3.2×	253

Table 2: GPU multiloop (fp32) vs. CPU (fp64) baseline, AMD Radeon 890M, driven spiking neurons, $K = 50\,000$ steps. "Sustained" is the warm steady-state dispatch; "single-run" is a cold-start run, ∼1.3–2× slower. End-to-end (incl. construction) speedup is 1.2–4.2×. Raw data: `experiments/data/campaign_warm_dispatch.csv`, `experiments/data/campaign_wallclock_summary.csv`, `experiments/data/campaign_reproducibility.csv`.

Numerical fidelity of the fp32 accelerator. `genesis/Scripts/benchmark/hh1952_ap_verify.g` compares the classic 1952 squid AP across CPU fp64, GPU per-step, and GPU multiloop paths: fp32 agrees with fp64 to ∼2×10^{-7} V over a single AP, all N neurons identical to $< 10^{-6}$ V (Fig. 2). Over long spiking runs, fp32/fp64 traces drift in spike *phase* but preserve waveform and firing rate, why timing uses driven neurons while correctness uses a single-AP window.

CUDA backend on NVIDIA hardware. The CUDA backend builds from the same source (`make USE_CUDA=1`) against the identical

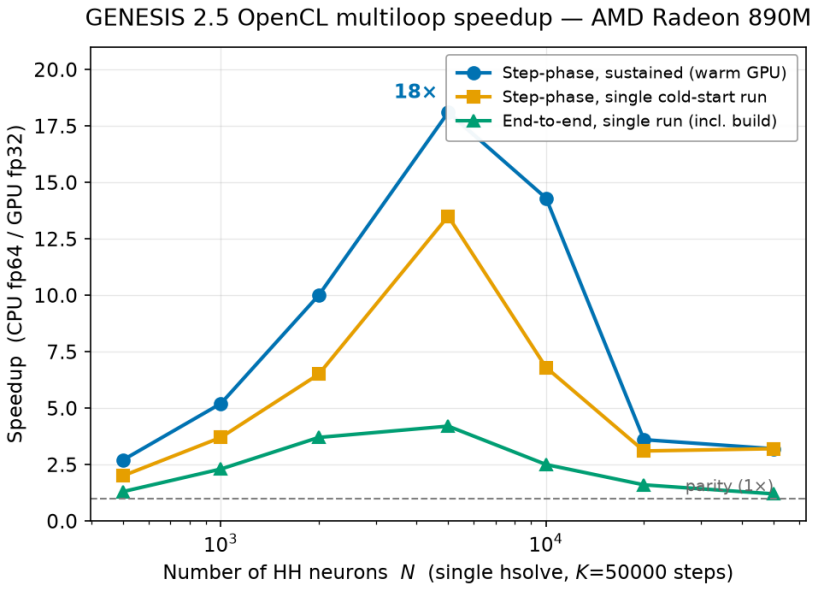

Figure 1: Measured wall-clock speedup, GPU multiloop (fp32) vs. CPU (fp64), sustained and cold-start step-phase, and end-to-end (single run). Both step-phase series peak near $N = 5000$ and fall beyond $N \approx 10\,000$ as the integrated GPU stops sustaining its fast clocks.

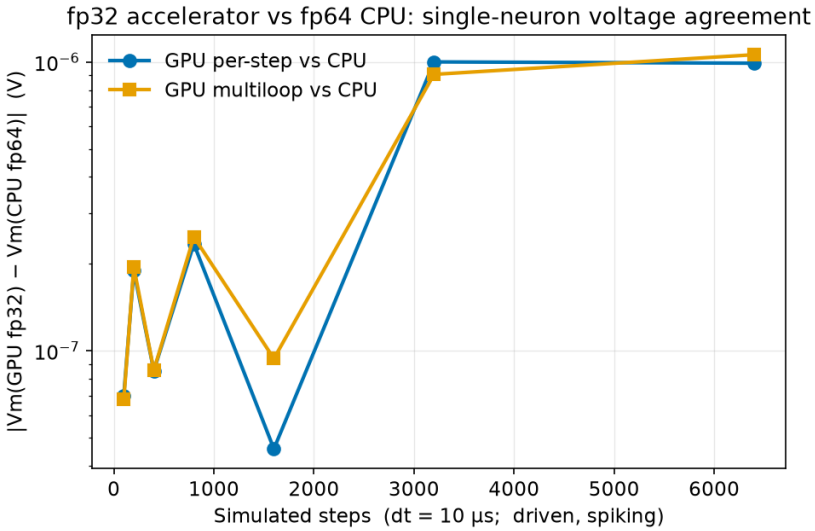

Figure 2: fp32 accelerator vs. fp64 CPU: absolute membrane-voltage difference for a single neuron as a function of simulated steps, for both the per-step and multiloop GPU paths.

harness, and reproduces the fp64 CPU action potential to 7×10^{-8} V, confirming parity with both the CPU and OpenCL paths. Table 3

reports a 10-replicate sweep on the two UMCS cluster GPUs.
These are *end-to-end* speedups: both arms are wall-clocked around the whole process, so construction, transfers and per-step host work are inside the measurement, and every figure in this paper uses that one definition. Timing the two arms by different clocks — a step-loop timer against a device-side kernel report — inflates the ratio by an order of magnitude and can invent apparent differences between cards; measured consistently the A40 and A100 are indistinguishable here ($21.0 \pm 0.3\times$ vs. $21.9 \pm 0.3\times$). Superseded figures and the reason they differ are recorded in `experiments/cuda_validation/RESULTS.md`.
The speedup grows with network size rather than peaking as the integrated GPU does (Table 2), and at $N = 500$ the accelerator barely pays for itself: at that size the fixed costs of construction and transfer dominate a run that the CPU finishes in under a second.

N	**NVIDIA A40**		**NVIDIA A100**	
	CPU (s)	**end-to-end**	**CPU (s)**	**end-to-end**
500	0.76	$1.5 \pm 0.1\times$	0.78	$1.8 \pm 0.3\times$
5 000	5.70	$8.7 \pm 0.3\times$	5.86	$9.7 \pm 0.4\times$
50 000	57.06	$\mathbf{21.0 \pm 0.3}\times$	58.47	$\mathbf{21.9 \pm 0.3}\times$

Table 3: CUDA backend (fp32) vs. CPU (fp64), single-compartment multiloop kernel (`hh_spiking_benchmark.g`, $K = 50\,000$ steps), UMCS cluster. Mean of 10 replicates; the quoted uncertainty propagates the standard deviations of both arms. **Both arms are timed identically**, by wall clock around the whole process, so the ratio is end-to-end and includes construction, transfers and per-step host work — see `cluster_bringup/53_singlecomp_walltime.sh`. The two cards are indistinguishable at every size. Raw data: `cluster_bringup/logs/singlecomp_walltime_inf02_20260815_132840.csv` (A40), `..._inf03_20260815_135147.csv` (A100).

Multi-compartment GPU acceleration. The multiloop kernel above is exact only for single-compartment (isopotential) neurons: for a real dendritic tree it silently drops all axial coupling. `hines_tree_eliminate` (Section 2.1) performs this elimination on-device, dispatched whenever the `hsolve` contains a multi-compartment tree.
Correctness. Linear-chain and branching benchmarks (the latter exercising sibling-array opcodes) compare GPU against fp64 CPU across `NCOMP` = 1–32, up to 4 branches and $N = 10\,000$ neurons: agreement is $\sim 10^{-7}$ V, matching single-compartment fidelity. One topology is not handled: a single-compartment branch off a branch point, which violates the kernel bootstrap's seed-row assumption. It had been documented but not enforced, so such a model was accelerated and returned wrong voltages silently; both backends now detect it at initialisation

and compute that `hsolve` on the CPU, on the same pattern used for unsupported opcodes.

Speed. Table 4 and Fig. 3 report speedup (mean ± std, 10 replicates) on the two UMCS cards, sweeping $N = 100$–$50\,000$ at `NCOMP` $= 16$, a range unreachable before the model-construction fix (Impact): $N = 50\,000$ took 23.8 ± 0.3 s to build there, versus not finishing before. Branching topology gives statistically indistinguishable speedups and is omitted from the figure. Dispatch uses $K = 200$ steps ($K = 100$ for $N \geq 5000$), smaller than the single-compartment sweeps' $K = 50\,000$.

Two figures per card answer different questions. The *step-phase* speedup times the simulation loop alone and measures what the kernel can do: both cards are slower than CPU at $N \leq 500$, pull ahead past $N \approx 500$–1000, and reach $37.3 \pm 0.1\times$ (A40) and $80.8 \pm 1.0\times$ (A100) at $N = 50\,000$, neither plateaued. The *end-to-end* speedup wall-clocks the whole process and is what a user experiences: it is far lower, $3.62 \pm 0.05\times$ (A40) and $3.90 \pm 0.07\times$ (A100) at $N = 50\,000$. The gap is not a kernel deficiency but Amdahl's law applied to a short run — at $K = 200$ steps the unaccelerated construction phase dominates, and the end-to-end figure climbs toward the step-phase ceiling as K grows. Quoting only the step-phase number would overstate what the accelerator delivers in practice.

The cards nearly coincide end-to-end despite the A100 leading by more than $2\times$ on the step phase — consistent with an fp32 kernel using no tensor cores, the GPU phases taking 7.83 s (A40) against 8.06 s (A100) at $N = 50\,000$. Raw data: `cluster_bringup/logs/weekend_campaign_*_20260816_clean.csv` and `..._multicomp_walltime_createmap_*_clean.csv`.

Parallelization granularity. `hines_tree_eliminate` assigns one GPU thread per tree, so speedup comes from spreading many neurons across threads, not from parallelizing elimination *within* a tree. For a single detailed morphology alone ($N = 1$), only one thread is active: measured directly (AMD Radeon 890M), the kernel is $\sim$20–$21\times$ *slower* than the CPU at `NCOMP` $= 2000$ and 8000, stable with tree size as expected for a single-thread bottleneck. The kernel therefore accelerates population-scale simulation (Table 4), not the single-detailed-cell regime.

Population-scale model construction was $O(n^2)$; now $O(n)$. Pushing toward the $\sim$31 000-neuron Blue Brain Project scale [9] exposed a defect independent of GPU support: 527 000 compartments did not finish building within 900 s. Profiling gave a fitted exponent of 2.31, traced to five pre-existing linear-scan-per-element patterns in element creation, path resolution and `hsolve` SETUP, none GPU-specific —

		step-phase		end-to-end	
N	**total comps**	**A40**	**A100**	**A40**	**A100**
100	1 600	0.2 ± 0.02	0.3 ± 0.02	0.43 ± 0.05	0.61 ± 0.02
500	8 000	1.1 ± 0.03	1.7 ± 0.11	0.77 ± 0.02	1.04 ± 0.03
1 000	16 000	2.0 ± 0.08	3.1 ± 0.07	1.10 ± 0.08	1.46 ± 0.04
2 000	32 000	4.5 ± 0.39	6.4 ± 0.18	1.37 ± 0.06	1.57 ± 0.05
3 000	48 000	6.7 ± 0.03	12.0 ± 0.37	1.87 ± 0.07	1.89 ± 0.04
5 000	80 000	6.9 ± 0.04	14.5 ± 0.47	2.60 ± 0.06	2.66 ± 0.03
7 000	112 000	9.8 ± 0.05	22.0 ± 0.43	2.92 ± 0.05	3.08 ± 0.03
10 000	160 000	13.7 ± 0.06	30.6 ± 0.68	3.01 ± 0.04	3.23 ± 0.04
15 000	240 000	20.3 ± 0.12	42.3 ± 0.77	3.27 ± 0.05	3.52 ± 0.03
20 000	320 000	25.3 ± 0.12	51.6 ± 1.16	3.39 ± 0.04	3.71 ± 0.04
30 000	480 000	31.9 ± 0.13	66.1 ± 1.12	3.53 ± 0.05	3.87 ± 0.05
40 000	640 000	35.2 ± 0.13	75.5 ± 0.96	3.56 ± 0.04	3.90 ± 0.04
50 000	800 000	37.3 ± 0.07	**80.8 ± 1.01**	3.62 ± 0.05	**3.90 ± 0.07**

Table 4: Multi-compartment GPU tree-elimination (fp32) vs. CPU (fp64), UMCS A40/A100, `NCOMP` = 16, linear-chain, batched multiloop ($K = 200$, $K = 100$ for $N \geq 5000$); mean ± std over 10 replicates. *Step-phase* times the simulation loop alone and measures kernel capability; *end-to-end* wall-clocks the whole process, including model construction, and is what a user experiences. The two arms of the end-to-end measurement are timed identically – see `cluster_bringup/54_multicomp_walltime.sh`. The end-to-end runs build the population with a single `createmap` from a prototype, as network models do, rather than the explicit SLI loop of the original benchmark; the step-phase figures are unaffected by that choice, since they exclude construction. "–" = not measured. Raw data: `cluster_bringup/logs/weekend_campaign_inf02_A40_20260816_clean.csv` and `..._inf03_A100_20260816_clean.csv` (step-phase), `cluster_bringup/logs/multicomp_walltime_createmap_inf02_A40_clean.csv` and `..._inf03_A100_clean.csv` (end-to-end).

each rescanning siblings, paths or compartments where hashing or amortized growth would do. Fixing all five (Table 5, Fig. 4) gives an exponent of 0.99: that population now builds in 14.6 ± 0.2 s, and 1.7 million compartments in 51.5±0.6 s. Output is byte-identical on every validation script, confirming a pure performance fix; full diagnosis in `genesis/src/hines/GPU_HINES_SOLVE_DESIGN.md`.

Speedup grows with simulation length. Model construction is a fixed one-time cost, so end-to-end speedup depends on run length. Sweeping K at fixed $N = 5000$ shows end-to-end speedup rising toward the step-phase ceiling, so the accelerator pays off most for the long runs typical of real modelling studies (`experiments/data/campaign_ksweep.csv`).

The multi-compartment figures are a floor set by the bench-

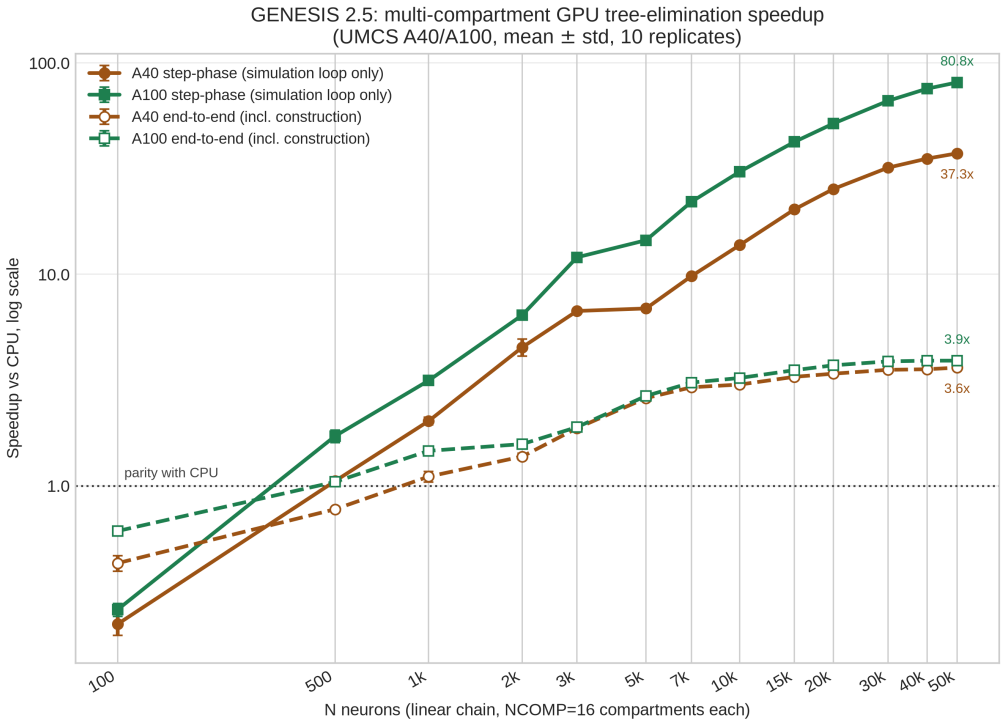

Figure 3: Multi-compartment GPU tree-elimination speedup vs. CPU (mean ± std, 10 replicates; log-log), UMCS A40/A100, sweeping N at `NCOMP` = 16 (linear-chain). Colour identifies the card, line style the metric: solid = step-phase (simulation loop only), dashed = end-to-end (whole process, including model construction). Both cards start below CPU parity and cross it near $N \approx 500$–1000. The step-phase curves keep climbing to 80.8× (A100) and 37.3× (A40) without plateauing, but the end-to-end curves flatten near 3.9× and 3.6×: at $K = 200$ steps the unaccelerated construction phase bounds the achievable speedup, and only the step-phase curves measure the kernel itself. The two cards nearly coincide end-to-end despite the A100's large step-phase lead, as expected for an fp32 kernel using no tensor cores.

mark, not a ceiling set by the kernel. The sweeps above use $K = 200$ ($K = 100$ for $N \geq 5000$), which at $dt = 50\,\mu$s is 5–10 ms of simulated activity, short enough that fixed costs dominate both arms. Repeating the measurement at $N = 10\,000$ over a range of K (Table 6) separates them: end-to-end speedup rises from 4.3× at $K = 200$ to $57.0 \pm 0.9\times$ at $K = 5000$, and step-phase from 13.7× to 116.2×. Both grow because the one-time costs they contain are amortized over more steps: model construction for the end-to-end figure, buffer setup and the single batched dispatch for the step-phase figure.

Fitting fixed and marginal costs separates them. Construction takes ≈1.3 s on both arms, and one step costs 2.31×10^{-2} s on the CPU

N	**total comps**	**before (s)**	**after: mean±std (s)**	**CV**
1 000	17 000	0.82	0.57 ± 0.08	13.2%
2 000	34 000	2.03	0.93 ± 0.003	0.3%
4 000	68 000	8.24	1.83 ± 0.04	1.9%
8 000	136 000	105.45	4.08 ± 0.03	0.7%
16 000	272 000	did not finish (60 s+)	7.54 ± 0.02	0.2%
31 000	527 000	did not finish (900 s+)	14.57 ± 0.18	1.2%
50 000	850 000	–	23.80 ± 0.32	1.3%
70 000	1 190 000	–	34.25 ± 0.44	1.3%
100 000	1 700 000	–	**51.48 ± 0.61**	1.2%

Table 5: Model-construction wall time (construction + `hsolve` SETUP + one step), `hh_branching_multicompartment_benchmark.g`, local machine, before vs. after the five fixes. "After": mean ± std, 3 replicates; "before": single replicate, not re-measured beyond $N = 8000$ ("–" = never attempted). Fitted exponent: 2.31 before, 0.99 after. Raw data: `experiments/data/construction_scaling_before_after.csv`.

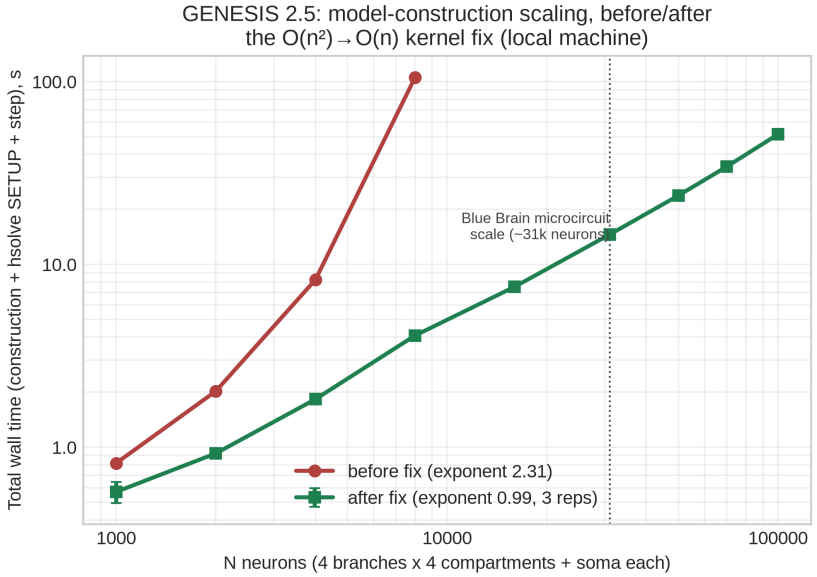

Figure 4: Model-construction wall time vs. N, log-log, before (single replicate) and after (mean ± std, 3 replicates) the five $O(n^2)$ fixes, extended to $N = 100\,000$. The "before" series stops at $N = 8000$: larger N did not finish within the tested budget.

against 1.42×10^{-4} s on the GPU, a ratio of 163×, with the device profile agreeing independently at 140.4 μs per step. At $K = 5000$ the GPU still reproduces the CPU voltages to $\sim 10^{-7}$ V, so this is amortization, not skipped work. Table 4 reports the short-K figures because they are what the standing benchmark measures, but a 250 ms simulation

of this population already reaches 57× end-to-end.

K steps	CPU (s)	GPU (s)	End-to-end
200	5.86	1.37	4.28 ± 0.12
500	13.00	1.42	9.18 ± 0.25
1 000	24.87	1.49	16.73 ± 0.36
2 000	48.03	1.63	29.48 ± 0.91
5 000	116.75	2.05	$\mathbf{57.01 \pm 0.88}$

Table 6: End-to-end wall clock against simulation length, $N = 10\,000$ neurons × 16 compartments (160 000 compartments), UMCS A40, mean ± std over 5 replicates, both arms timed identically. The fixed cost each arm carries, ≈1.3 s of model construction, is unchanged across the sweep, so the speedup rises as K grows: at $K = 200$ construction is most of the GPU run, at $K = 5000$ it is under a tenth of it. Raw data: `cluster_bringup/logs/multicomp_ksweep_inf02_A40_20260816.csv`, produced by `cluster_bringup/55_multicomp_ksweep.sh`.

Reproducing the PGENESIS scaling result. The existing MPI path is unaffected; a strong-scaling sweep ($N = 2400$ HH neurons, $P = 1$–24 ranks, 12-core/24-thread host) gives near-ideal scaling at $P = 2$ ($E_P = 0.95$), an efficiency knee near $P \approx 5$, and a recommended $P = 4$–8 (3.1–4.0×, 50–78% efficiency) for this problem size.

Head-to-head against NEURON and CoreNEURON. The speedups above are internal (GPU vs. CPU within GENESIS) and say nothing about how GENESIS compares with other simulators. We therefore ran the Vogels–Abbott COBAHH network [15, 16] in GENESIS (`Scripts/VAnet2`) and in the reference NEURON implementation (ModelDB 83319, `NEURON/cobahh`) on the same cluster node, all arms single-threaded on CPU (Table 7, Fig. 5). GENESIS 2.5 completes the 5.0 s simulation in 46.9 ± 2.1 s, 1.63 ± 0.07× faster than CoreNEURON and 2.04 ± 0.09× faster than NEURON 9.0.2. In throughput terms the run advances 4×10^8 neuron-steps, so GENESIS sustains 8.5×10^6 neuron-steps/s against 5.2×10^6 for CoreNEURON and 4.2×10^6 for NEURON.

CoreNEURON exists to make NEURON models faster: it discards the interpreter and rebuilds the data layout for vectorisation. The GENESIS arm measured here uses no accelerator at all, only the plain CPU solver. CoreNEURON is in turn 1.25× faster than NEURON 9.0.2, the expected ordering, which suggests that arm is configured correctly.

Because these are independent implementations, equivalence was established rather than assumed: both networks have 4000 single-compartment cells (3200 excitatory, 800 inhibitory) with 80 synapses each, both are driven externally for only the first 50 ms and then run on recurrent activity, and both use $dt = 0.05$ ms for 5.0 s. Critically, both settle

into the same regime, 26.8 Hz mean rate in GENESIS against 27.9 Hz in NEURON, so neither does materially less synaptic work. Obtaining the GENESIS rate required instrumenting every spikegen. The benchmark records only three cells, and two sit at the grid edge where they are underconnected, firing at 7.6 and 0.6 Hz against 15.8 Hz centrally. The stock benchmark cannot run under CoreNEURON at all: its channels are ChannelBuilder objects rebuilt inside the interpreter, so CoreNEURON aborts in `nrn_setup`. We transcribed them into NMODL, exactly reproducing the published COBAHH rates at $V_T = -63$ mV, after which CoreNEURON yields a spike train byte-identical to NEURON's. Harness, mechanisms and full method are in `cluster_bringup/coreneuron/`. This comparison establishes a single-core baseline only. CoreNEURON targets GPU and multi-rank MPI execution, and exercises neither here; the GENESIS arm is likewise its CPU arm, since VAnet2 cannot yet use the accelerator efficiently (Section 2.1). We attempted the GPU configuration and could not reach it on this cluster. CoreNEURON offloads through OpenACC, so it needs NVIDIA's compilers: built from source with NVHPC 24.11 and 25.3, NEURON 9.0.2 compiles and links, and our transcribed channels pass its GPU code generator, but the binaries segfault: both code generators, the Python module, and the interpreter on a single passive-soma model. The container route was closed by cluster policy (no subuid range, no GPU container toolkit). We report no GPU figure for CoreNEURON because we could not obtain one we would trust, and note the contrast with the effort the comparison asks of a user: GENESIS 2.5 enables its accelerator with a build flag and a run-time setting.

Simulator	**Channels**	**Wall (s)**	**Spikes**
GENESIS 2.5 (VAnet2)	tabulated	**46.9 ± 2.1**	536 600
CoreNEURON 9.0.2	compiled NMODL	76.5 ± 0.3	558 824
NEURON 9.0.2	compiled NMODL	95.8 ± 0.2	558 824
NEURON 9.0.2	ChannelBuilder	123.3	574 138
NEURON 8.0.2	ChannelBuilder	76.5	–

Table 7: Vogels–Abbott COBAHH network, 4000 single-compartment cells, 80 synapses per cell, $dt = 0.05$ ms, 5.0 s simulated, UMCS node inf03, all arms single-threaded CPU. Mean ± std over 3 replicates for the top three rows; the two ChannelBuilder rows are single runs, shown for reference only. GENESIS 2.5 is $1.63 \pm 0.07\times$ faster than CoreNEURON. The two simulators reach the same activity regime (26.8 vs. 27.9 Hz), so the wall times reflect comparable work. All NEURON and CoreNEURON replicates returned an identical spike count, confirming the runs are deterministic. NEURON 8.0.2 is included because its pip distribution ships the CoreNEURON Python interface but not the engine, so the CoreNEURON arm requires NEURON 9. Harness: `cluster_bringup/coreneuron/`.

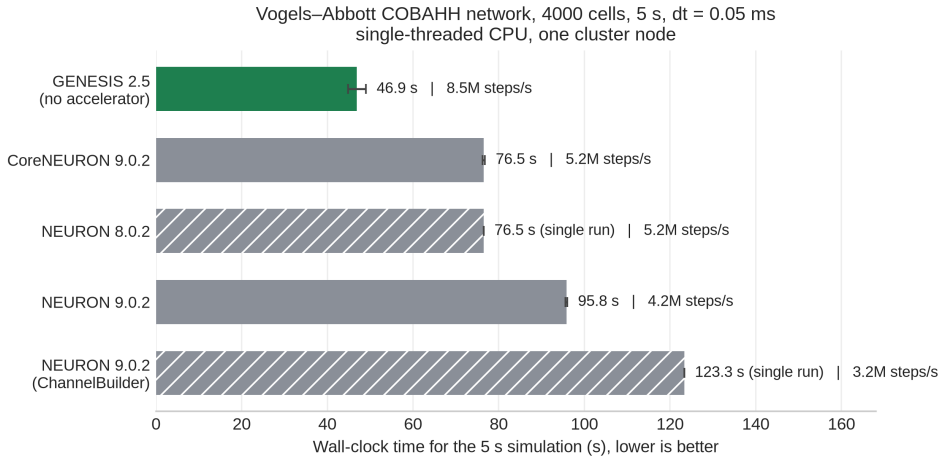

Figure 5: GENESIS 2.5 against NEURON and CoreNEURON on the Vogels–Abbott COBAHH network, all arms single-threaded CPU on one UMCS node. Hatched bars are single runs; the others are means of 3 replicates with standard deviations. Throughput is quoted over the 4×10^8 neuron-steps the 5 s simulation advances. The GENESIS arm uses no accelerator, only the plain CPU solver, yet completes the network in two thirds of CoreNEURON's time.

Full replication steps and raw data are in `paper/docs/REPLICATION.md`.

4. Impact

   GENESIS/PGENESIS users with channel-kinetics-heavy models now have a path to GPU acceleration (Section 3) without migrating models, SLI scripts, or MPI workflows. That is the principal barrier an architecturally different successor such as MOOSE would otherwise impose.

   The three Hines-solver defects fixed along the way (Section 2) are independent of GPU support and affect any multi-neuron `hsolve` configuration: before the fix, `inject`-driven voltage transients were silently suppressed for every neuron but the first, with no error raised. That defect predates this release and is now fixed for any multi-neuron `hsolve` user, whether or not they use an accelerator.

   A broken $O(N)$ scheduler loop, re-scheduling neurons $2 \ldots N$ on every step, is fixed; that repair is what makes Section 3's throughput attainable at all.

   The accelerator has a deliberate scope. Both kernels interpret the `hsolve` opcode program directly and implement the five opcodes a `tabchannel` model driven by current injection produces. Compartments needing anything else (synaptic channels, a `spike` element, GHK, concentration pools) are detected at `SETUP` and computed on the CPU. Validating this on a published network model exposed a fourth latent

defect: the detection existed but its result was never consulted, so such compartments reached a kernel that mis-parses their opcode stream, giving wrong voltages from the first step with no error raised. It affected none of the benchmarks here, since none uses those elements, but it affects any real network model that does. The check is now enforced, and the boundary it draws defines what this release accelerates.

Pushing toward the Blue Brain Project's scale [9] turned up a defect entirely separate from the GPU work: five pre-existing linear-scan patterns compounding into $O(n^2)$ model-construction time (Section 3). None is accelerator-specific, so fixing all five benefits any GENESIS/PGENESIS user building a large model, whether or not an accelerator is available.

5. Conclusions

GENESIS 2.5 adds opt-in OpenCL and CUDA acceleration to GENESIS 2.4/PGENESIS 2.4 for Hodgkin–Huxley `tabchannel` models, without changes to model files, SLI scripts, or MPI workflows (Section 3 reports speedup and fidelity for both integrated-GPU and cluster-class hardware). Beyond the single-compartment multiloop kernel, `hines_tree_eliminate` extends acceleration to axially-coupled multi-compartment trees at population scale; the single-detailed-cell regime is not helped by this design, which we measure and report.

The release also carries four correctness fixes independent of accelerator use, benefiting any GENESIS/PGENESIS user. Three are latent Hines-solver defects that silently suppressed `inject`-driven voltage transients in any multi-neuron `hsolve`. The fourth is in the accelerator dispatch itself: compartments using opcodes the kernels do not implement were sent to the GPU anyway, producing silently wrong results; they are now detected and computed on the CPU. Separately, five pre-existing linear-scan patterns compounding into $O(n^2)$ model-construction time are fixed, restoring linear scaling and making population-scale models buildable at all.

Measured against other simulators on a published network model, GENESIS 2.5 is $1.63 \pm 0.07\times$ faster than CoreNEURON and $2.04 \pm 0.09\times$ faster than NEURON 9.0.2 for single-threaded CPU execution of the Vogels–Abbott COBAHH benchmark, after verifying that the two implementations are equivalent (Section 3).

Future work. Four directions follow directly from the limits measured here, in what we judge to be descending order of value to users.

Scale. The kernel assigns one thread per tree, so it accelerates population-scale simulation and not the single detailed cell, where only one thread is active and the GPU is $\sim 20\times$ slower than the CPU. Intra-tree paral-

lelism — cyclic reduction or a comparable parallel tridiagonal scheme — would extend acceleration to the detailed-morphology regime that much of the compartmental-modelling literature works in. This is the dimension along which the present design is furthest from complete.

Coverage. Synaptically coupled networks are currently declined rather than accelerated, because the kernels implement only the opcodes a `tabchannel` model driven by current injection produces. Extending coverage to synaptic channels, spike generators and concentration pools would bring the accelerator to the models most GENESIS users actually run. A structural obstacle sits behind this: GENESIS associates one spike generator with each `hsolve`, so a spiking network is built as one solver per cell, and per-solver dispatch overhead then dominates. Batching dispatch across `hsolve` elements is the corresponding fix, and it interacts with event ordering, since a spike raised inside a batch cannot reach another cell's synapses before the batch ends.

Comparison. The head-to-head reported here is single-threaded CPU on one node. CoreNEURON is designed for GPU and multi-rank MPI execution, and Arbor for many-core hardware; a comparison across those configurations, and against Arbor on a network reimplemented with the same care taken here, would place GENESIS 2.5 more completely.

Precision. The kernels are fp32 throughout, which is what lets them run on integrated GPUs lacking double precision, and which costs $\sim 10^{-7}$ V against the fp64 CPU over the runs measured. Whether that divergence stays bounded over the much longer runs used in plasticity or development studies is not established here and should be characterised before such models rely on it.

Packaging into a container-based deployment pipeline [17] is in progress.

Acknowledgements

The authors thank the Maria Curie-Skłodowska University (UMCS) in Lublin and the LubMAN UMCS computing centre for access to the "Lunar" cluster (NVIDIA A100 and A40 GPU nodes) at the UMCS Institute of Mathematics, commissioned in 2015 [18] and since upgraded with the GPU nodes used in this work, and Karol S. Karpowicz and Prof. Przemysław Stpiczyński for provisioning and support of the compute environment on which the CUDA backend was built and validated. The authors also thank WarsawIQ for the AMD Radeon 890M integrated GPU used for the laptop-class benchmarks reported here.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System*. Springer, 1998.

[2] GENESIS 3 Development Plans. `http://www.genesis-sim.org/GENESIS/G3/G3-dev-plans.html`

[3] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15.

[4] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.

[5] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3.

[6] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.

[7] Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019, pp. 274–282. doi:10.1109/EMPDP.2019.8671560.

[8] Kumbhar P, Hines M, Fouriaux J, Ovcharenko A, King J, Delalondre F, Schürmann F. CoreNEURON: An Optimized Compute Engine for the NEURON Simulator. *Frontiers in Neuroinformatics* 2019, 13:63. doi:10.3389/fninf.2019.00063.

[9] Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456–492. doi:10.1016/j.cell.2015.09.029.

[10] Hines M. Efficient computation of branched nerve equations. *International Journal of Bio-Medical Computing* 1984, 15(1):69–76. doi:10.1016/0020-7101(84)90008-4.

[11] Hines ML, Carnevale NT. The NEURON Simulation Environment. *Neural Computation* 1997, 9(6):1179–1209. doi:10.1162/neco.1997.9.6.1179.

[12] Gewaltig M-O, Diesmann M. NEST (NEural Simulation Tool). *Scholarpedia* 2007, 2(4):1430. doi:10.4249/scholarpedia.1430.

[13] Stimberg M, Brette R, Goodman DFM. Brian 2, an intuitive and efficient neural simulator. *eLife* 2019, 8:e47314. doi:10.7554/eLife.47314.

[14] Yavuz E, Turner J, Nowotny T. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports* 2016, 6:18854. doi:10.1038/srep18854.

[15] Vogels TP, Abbott LF. Signal Propagation and Logic Gating in Networks of Integrate-and-Fire Neurons. *Journal of Neuroscience* 2005, 25(46):10786–10795. doi:10.1523/JNEUROSCI.3508-05.2005.

[16] Brette R, Rudolph M, Carnevale T, Hines M, Beeman D, Bower JM, et al. Simulation of networks of spiking neurons: A review of tools and strategies. *Journal of Computational Neuroscience* 2007, 23(3):349–398. doi:10.1007/s10827-007-0038-6.

[17] Chlasta K, Sochaczewski P, Wójcik GM, Krejtz I. Neural simulation pipeline: Enabling container-based simulations on-premise and in public clouds. *Frontiers in Neuroinformatics* 2023, 17:1122470. doi:10.3389/fninf.2023.1122470.

[18] Maria Curie-Skłodowska University (UMCS), Instytut Matematyki. Superkomputer LUNAR uruchomiony w Instytucie Matematyki UMCS. `https://www.umcs.pl/pl/aktualnosci,57,superkomputer-lunar-uruchomiony-w-instytucie-matematyki-umcs,27879.chtm`, 2015.

Current executable software version

Not applicable; this release is distributed as source code only (see Current code version above).